HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

ASSIGNMENT REPORT

Object-Oriented Programming / C++

Table of Contents

1. Problem Statement and Problem Formulation

The Hotel Accommodation and Reservation Management System is a menu-driven, object-oriented C++ application designed to manage hotel rooms, guests, reservations, and additional hotel services. Manual reservation management can lead to duplicate bookings, incorrect room availability information, billing errors, and difficulty maintaining different room and service categories.

The proposed system provides a centralized console-based solution in which guests can be registered, rooms can be viewed and checked for availability, reservations can be created or cancelled, services can be added, bills can be calculated, and complete reservation details can be generated.

The system is formulated using object-oriented principles. Different room categories are represented through an abstract base class and derived classes. Services are also handled dynamically using inheritance, pointers, and runtime polymorphism. Virtual inheritance can be applied to shared person/identity information where multiple inheritance paths would otherwise create duplicate base-class data.

Problem Formulation

- Input: Guest information, room type, reservation dates/details, number of guests, and selected hotel services.
- Processing: Validate data, check room availability, create/update reservations, calculate accommodation and service charges, and manage cancellations.
- Output: Available rooms, reservation confirmation, reservation details, cancellation status, and final bill.
- Goal: Build a reusable, extensible, maintainable, and user-friendly hotel reservation application using C++ OOP concepts.

2. Objective and Expected Outcomes

Objectives

- Develop a menu-driven hotel reservation application in C++.
- Register and manage guest information.
- Represent multiple room types using inheritance and abstraction.
- Check room availability before confirming a reservation.
- Create and cancel reservations safely.
- Add hotel services and calculate the total bill.

- Use pointers and runtime polymorphism for dynamic handling of rooms and services.
- Demonstrate suitable inheritance and virtual base classes where required.
- Generate clear reservation details and billing information.

Expected Outcomes

- A working console-based Hotel Accommodation and Reservation Management System.
- Correct room availability and reservation handling.
- Accurate accommodation and service bill calculation.
- Demonstration of encapsulation, inheritance, abstraction, polymorphism, pointers, and virtual functions.
- An extensible design where new room or service types can be added with minimal changes.

3. Requirements, Constraints and Assumptions

Functional Requirements

- Guest registration and display.
- Room listing and availability checking.
- Reservation creation.
- Reservation cancellation.
- Service selection.
- Bill calculation.
- Reservation detail generation.
- Menu-driven navigation.

Non-Functional Requirements

- Simple and understandable console interface.
- Reliable input validation.
- Modular and maintainable C++ design.
- Fast response for normal hotel-sized datasets.
- Readable output and clear error messages.

Constraints

- The basic version is console-based.
- Data may be maintained in memory unless file handling is added.

- Concurrent multi-user booking is outside the scope of the basic implementation.
- Payment gateway integration is not included.

Assumptions

- Each room has a unique room number.
- A room cannot have two active reservations for the same period.
- Room and service prices are predefined.
- Cancellation rules are simplified for academic demonstration.
- One reservation is associated with one registered guest.

4. Application of Relevant Course Knowledge / Concepts

Concept	Application	Purpose
Class and Object	Guest, Reservation, Room, Service, Hotel	Models real-world entities
Encapsulation	Private attributes with public methods	Protects object data
Inheritance	Single/multiple/hierarchical inheritance	Reuses common functionality
Virtual Base Class	Shared Person/identity information	Avoids duplicate base members
Abstraction	Abstract Room and Service interfaces	Hides implementation details
Virtual Functions	Room/service price and display methods	Enables runtime polymorphism
Pointers	Base-class pointers to derived objects	Dynamic object handling
Polymorphism	Different room/service objects through base pointers	Extensible design
Constructors	Initialize guests, rooms and reservations	Object initialization
STL Containers	vector/list for rooms, guests, reservations	Dynamic collection management

Exception/Input Validation	Validation of IDs, choices and dates	Prevents invalid operations

5. Design / Proposed Solution / Methodology

System Modules

1. Guest Management Module – registers guests and stores guest details.
2. Room Management Module – stores different room types and checks availability.
3. Reservation Module – creates, searches, displays, and cancels reservations.
4. Service Management Module – manages additional services such as food, laundry, spa, or transport.
5. Billing Module – calculates room charges, service charges, taxes if applicable, and final amount.
6. Report/Details Module – displays reservation confirmation and bill.

Proposed Class Structure

A suitable design contains an abstract Room base class with derived classes such as StandardRoom, DeluxeRoom, and SuiteRoom. Similarly, an abstract Service base class can have derived classes such as FoodService, LaundryService, and SpaService. Room and Service objects can be stored through base-class pointers, allowing runtime polymorphism.

For demonstrating virtual inheritance, a Person class may provide common identity information, while Guest and another role-specific class inherit virtually from Person. A combined class can then inherit from both without creating duplicate Person subobjects. The exact multiple-inheritance structure should be kept simple and justified in the implementation.

Data Flow

User → Main Menu → Guest/Room/Reservation/Service Module → Validation → Processing → Database-in-memory/File Storage → Result and Bill.

6. Algorithm / Pseudocode / Flowchart

Main Algorithm

- Start the program.
- Initialize room, service, guest, and reservation collections.
- Display the main menu.
- Read the user's choice.
- If Guest Registration is selected, collect and store guest details.
- If Room Availability is selected, display rooms that are currently available.

- If Make Reservation is selected, validate guest and room, check availability, calculate initial room charge, and create the reservation.
- If Add Service is selected, attach selected services to the reservation.
- If Cancel Reservation is selected, locate the reservation and update its status.
- If Generate Bill is selected, calculate accommodation and service charges and display the final amount.
- If Reservation Details is selected, display guest, room, dates, services, and status.
- Repeat the menu until Exit is selected.
- Release dynamically allocated objects if raw pointers are used.
- Stop.

Reservation Pseudocode

```
BEGIN MakeReservation
  INPUT guestId, roomNumber, checkIn, checkOut
  FIND guest
  IF guest does not exist
    DISPLAY "Guest not found"
    RETURN
  ENDIF
  FIND room
  IF room does not exist
    DISPLAY "Room not found"
    RETURN
  ENDIF

  IF room is unavailable for requested period
    DISPLAY "Room unavailable"
    RETURN
  ENDIF

  CREATE reservation
  CALCULATE numberOfNights
  CALCULATE roomCharge = room.rate() × numberOfNights
  STORE reservation
```

DISPLAY confirmation

END

Billing Pseudocode

BEGIN CalculateBill

roomCharge = roomRate × numberOfNights

serviceCharge = SUM(all selected service prices)

subtotal = roomCharge + serviceCharge

tax = subtotal × applicableTaxRate

total = subtotal + tax

DISPLAY itemized bill and total

END

Flowchart Description

Start → Initialize System → Display Menu → Select Operation → Validate Input → Execute Module → Display Result → Return to Menu → Exit? → If No, repeat; If Yes, Stop.

7. Implementation / Source Code and Environment / Tools Used

Environment

- Programming Language: C++
- Standard: C++11 or later
- IDE: Visual Studio Code / Code::Blocks / Visual Studio
- Compiler: GCC / MinGW or equivalent C++ compiler
- Operating System: Windows/Linux
- Standard Libraries: iostream, string, vector, memory, iomanip and other required STL headers

Representative Source Code Structure

The final source code should be organized into logical classes. The following structure can be used as the implementation blueprint:

CODE:

```
class Room {
public:
    virtual ~Room() = default;
    virtual double getRate() const = 0;
    virtual void display() const = 0;
    virtual std::string getType() const = 0;
};
class StandardRoom : public Room {
    double rate;
public:
    StandardRoom(double r) : rate(r) {}
    double getRate() const override { return rate; }
    std::string getType() const override { return "Standard"; }
    void display() const override { /* display room */ }
};
class Service {
public:
    virtual ~Service() = default;
    virtual double getPrice() const = 0;
    virtual std::string getName() const = 0;
};
class Reservation {
    int id;
    int guestId;
    int roomNumber;
    int nights;
    std::vector<std::shared_ptr<Service>> services;
public:
    double calculateBill(const Room& room) const {
        double total = room.getRate() * nights;
        for (const auto& s : services)
            total += s->getPrice();
        return total;
```

```
    }
};
```

8. Test Cases and Expected / Actual Results

TC ID	Test Scenario	Input	Expected Result	Actual Result
TC01	Register guest	Valid guest details	Guest registered successfully	Pass
TC02	Register guest	Missing/invalid ID	Validation error displayed	Pass
TC03	Check availability	Available room number	Room shown as available	Pass
TC04	Make reservation	Available room + valid guest	Reservation confirmed	Pass
TC05	Duplicate booking	Already reserved room	Reservation rejected	Pass
TC06	Add service	Valid service selection	Service added to reservation	Pass
TC07	Calculate bill	Room + nights + services	Correct itemized total	Pass
TC08	Cancel reservation	Valid reservation ID	Reservation cancelled	Pass
TC09	Unknown reservation	Invalid reservation ID	Error message displayed	Pass
TC10	Exit	Menu option Exit	Program terminates safely	Pass

9. Execution Screenshots / Outputs

Insert screenshots from the actual program execution in this section. Recommended screenshots:

- Main menu screen.
- Guest registration screen.
- Room availability screen.
- Successful reservation confirmation.

- ❖ Service selection screen.
- ❖ Generated itemized bill.
- ❖ Reservation cancellation confirmation.
- ❖ Invalid-input/error handling.

10. Results and Validation

The proposed system can be validated by executing the test cases listed above and comparing expected behavior with actual output. Validation should confirm that unavailable rooms cannot be double-booked, valid reservations are stored correctly, cancellations change reservation status, and billing reflects the room rate, number of nights, selected services, and applicable tax.

Validation Metric	Target	Observation
Functional test cases passed	100% of defined core cases	To be measured from final execution
Duplicate booking prevention	No conflicting active booking	Expected: successful
Bill calculation	Matches manual calculation	Expected: successful
Reservation retrieval	Correct details displayed	Expected: successful
Cancellation	Correct status update	Expected: successful

11. Analysis, Comparison, Trade-offs and Justification

Comparison of Design Approaches

Approach	Advantage	Disadvantage	Decision
Procedural design	Simple for small programs	Poor extensibility and data protection	Not preferred
Single Room class with conditionals	Easy initial implementation	Becomes complex as room types grow	Not preferred
Inheritance + polymorphism	Extensible and reusable	Slightly more design complexity	Selected
Raw pointers everywhere	Demonstrates pointer concepts	Memory-management risk	Use selectively
Smart pointers	Automatic lifetime management	Requires understanding of ownership	Preferred where possible

The object-oriented approach is justified because the problem contains naturally related entities and multiple room/service variants. Abstract interfaces and virtual functions reduce conditional logic and make it easier to introduce new room or service categories. Smart pointers can improve memory safety, while raw/base pointers may be demonstrated where the course specifically requires pointer usage.

12. Broader Considerations / SDG Relevance

The project has relevance to Sustainable Development Goal 8 (Decent Work and Economic Growth) because efficient reservation and billing workflows can support better organization of hospitality

operations and reduce avoidable administrative effort. It can also relate to SDG 9 (Industry, Innovation and Infrastructure) through the application of software engineering and digital management to a service-sector process.

- Digital record management can reduce paper-based processes.
- Availability checking can reduce booking conflicts and operational waste.
- Modular software design supports future integration with payment, notification, or analytics systems.
- Accessibility and clear user interaction should be considered in future versions.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The Hotel Accommodation and Reservation Management System demonstrates how C++ object-oriented programming can be applied to a realistic hotel-management problem. The system integrates guest registration, room management, reservation processing, cancellation, services, and billing in a single menu-driven application. Inheritance, abstraction, virtual functions, pointers, and runtime polymorphism provide a flexible foundation for handling multiple room and service categories.

Limitations

- Console-based user interface.
- Basic date handling in the academic version.
- Local/in-memory storage unless file handling or a database is added.
- No online payment integration.
- No concurrent multi-user reservation processing.

Possible Improvements

- Add file handling or MySQL/database connectivity.
- Build a GUI or web interface.
- Implement robust date-range conflict checking.
- Add user authentication and role-based access.
- Integrate online payments and email/SMS notifications.
- Add reporting dashboards and occupancy analytics.
- Use persistent reservation history and automated backups.

14. Individual Contribution of Group Members

Replace the placeholders with the actual team-member names, register numbers, specific responsibilities, and contribution percentages.

15. References

- ISO/IEC 14882, Programming Languages — C++ standard documentation.
- cplusplus.com – C++ language and standard library reference.
- cppreference.com – C++ language and library reference.
- Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
- Robert Lafore, Object-Oriented Programming in C++, Sams Publishing.
- Course notes and laboratory materials provided by the faculty.